Principles of Robot Autonomy I

Planning and Control: Trajectory optimization, trajectory planning



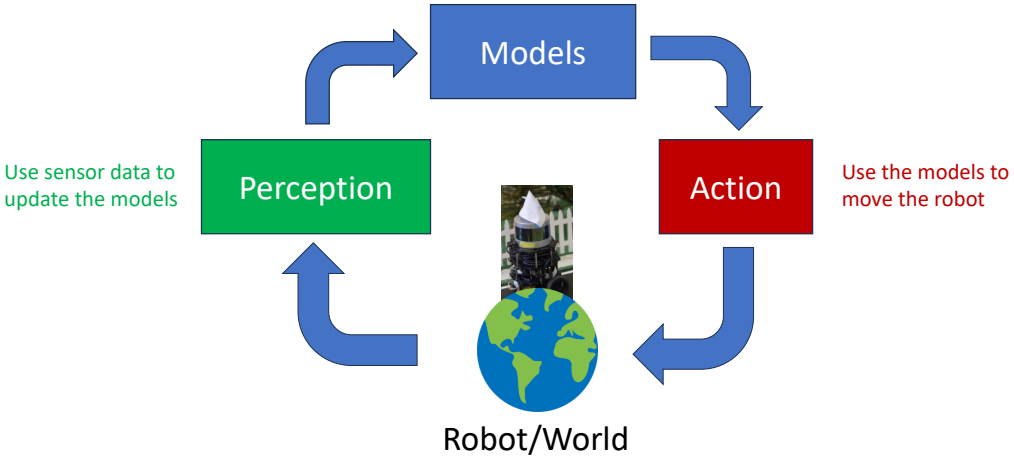






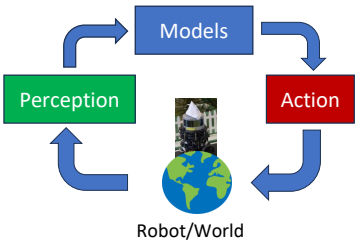


Full Stack Autonomy: the Perception-Action Loop



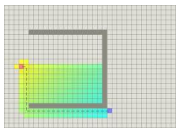
Action: Planning and Control Stack

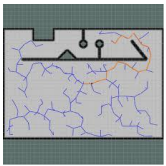
Use the models to move the robot



Path planning:

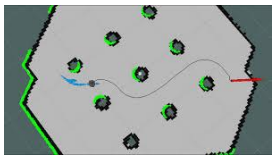
- A* planning on a graph
- RRT, RRT*





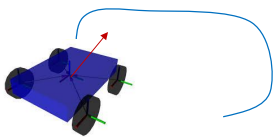
Trajectory optimization:

- Optimal control
- Numerical solvers



Trajectory following:

- Kinematic/dynamic models
- PID control
- LQR, gain scheduled LQR



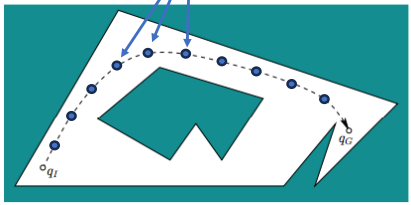
Today’s trajectory

- **Where it fits:** from a path plan to an executable motion in the autonomy stack.
- **What we optimize:** states, controls, costs, dynamics, and constraints.
- **How we solve it:** an optimization primer; shooting and collocation.
- **How we make it fast:** LQR, iLQR, and solvers that exploit time structure.
- **How we use it:** feedback through MPC; capabilities and limitations.

Aim: understand the ideas well enough to formulate the problem, various solution strategies, and choose appropriate numerical tools.

Trajectory Optimization





Trajectory optimization problem:

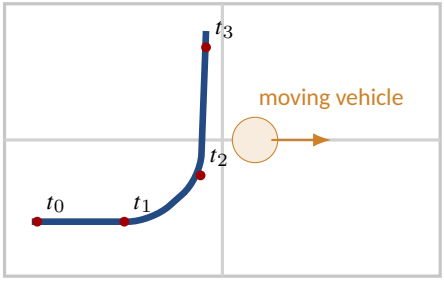
$$\min_{u_{0:T}} \sum_{t=0}^{T} c_t(x_t, u_t)$$ Traj. cost

$$\text{s.t.} \quad x_{t+1} = f(x_t, u_t)$$ Dynamic constraints

$$x_t \in \mathbb{X}_{\text{Free}}$$ Collision constraints

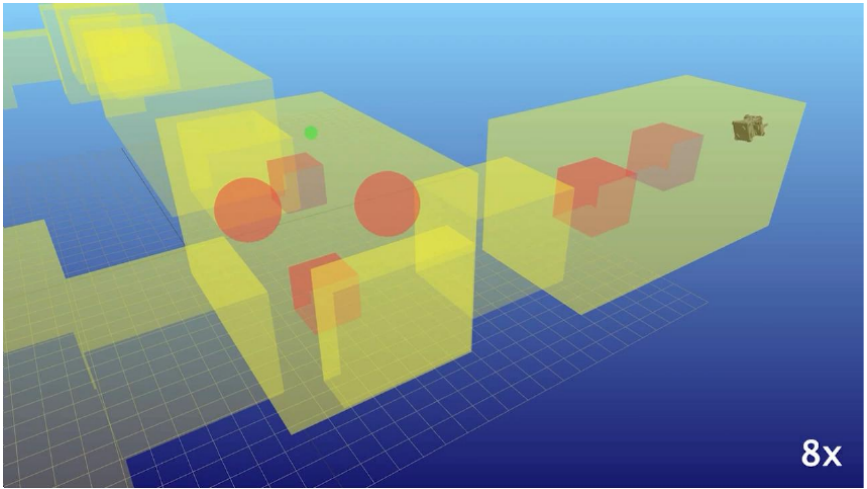
Goal: Find a dynamically feasible trajectory (close to the path) so the robot doesn't hit things!

Trajectory: A timed, dynamically feasible motion



- A path specifies geometry: $p(s)$.
- A trajectory specifies **when**: $x(t)$ and the inputs $u(t)$ that produce it.
- Example state: position, heading, speed. Inputs: acceleration and steering.
- Timing changes acceleration, feasibility, and encounters with other vehicles.

Trajectory Optimization for an AstroBee Flying inside the ISS



Trajectory Optimization for a drone flying through a NeRF environment map

Adamkiewicz, M., Chen, T., Caccavale, A., Gardner, R., Culbertson, P., Bohg, J. and Schwager, M., Vision-only robot navigation in a neural radiance world. *IEEE Robotics and Automation Letters*, 2022.



Trajectory planning for a high-speed car

Sampling-based kinodynamic planning: SST and SST*

Li, Yanbo, Zakary Littlefield, and Kostas E. Bekris, IJRR, 2016.



Click image to play video

Dynamic manipulation beyond quasi-static motion

Admissible Velocity Propagation (AVP)

Quang-Cuong Pham, Stéphane Caron, Puttichai Lertkultanon, Yoshihiko Nakamura. IJRR 2017.



Click image to play video

Optimal control problem

$$\min_{x(\cdot),u(\cdot)} \quad \underbrace{\int_0^T c(x(t),u(t))\,dt}_{\text{stage/running cost}} + \underbrace{h(x(T))}_{\text{terminal cost}}$$

subject to

$$\dot{x}(t) = f(x(t),u(t)), \qquad \text{dynamic/kinematic constraints}$$

$$x(0) = x_0, \qquad \text{initial condition}$$

$$x(t) \in \mathcal{X}_{\text{free}}(t), \quad u(t) \in \mathcal{U}, \qquad \text{safety and system limits}$$

Optimal control problem

$$\min_{x(\cdot),u(\cdot)} \quad \underbrace{\int_0^T c(x(t),u(t))\,dt}_{\text{stage/running cost}} + \underbrace{h(x(T))}_{\text{terminal cost}}$$

subject to	$\dot{x}(t) = f(x(t),u(t)),$	dynamic/kinematic constraints
	$x(0) = x_0,$	initial condition
	$x(t) \in \mathcal{X}_{\text{free}}(t), \quad u(t) \in \mathcal{U},$	safety and system limits

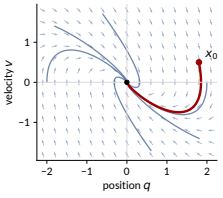
- A terminal *cost* expresses a preference; a required endpoint needs a terminal *constraint*.
- **Infinite dimensional:** continuous functions of time, $x(\cdot)$ and $u(\cdot)$.
- **Difficult in general:** No general closed-form solution. Numerical approximations and iterative methods are needed. Local minima are common.

Further study: Stanford AA203. D. E. Kirk, *Optimal Control Theory: An Introduction* (Dover, 2004). D. Liberzon, *Calculus of Variations and Optimal Control Theory: A Concise Introduction* (2012).

Two forms of optimal control solution

Closed loop: a policy over state space

$$u^*(t) = \pi(x(t), t)$$

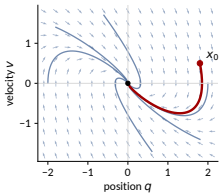


Optimal action for each (x, t). General nonlinear problems face the curse of dimensionality.

Two forms of optimal control solution

Closed loop: a policy over state space

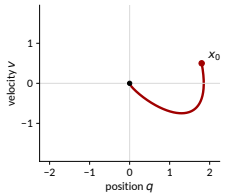
$$u^*(t) = \pi(x(t), t)$$



Optimal action for each (x, t). General nonlinear problems face the curse of dimensionality.

Open loop: one specified initial state

$$u^*(t) = u^*(t; x_0)$$

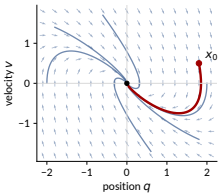


One nominal trajectory $(x_d(t), u_d(t))$. Models, disturbances, and sensing errors can cause deviation.

Two forms of optimal control solution

Closed loop: a policy over state space

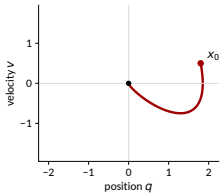
$$u^*(t) = \pi(x(t), t)$$



Optimal action for each (x, t). General nonlinear problems face the curse of dimensionality.

Open loop: one specified initial state

$$u^*(t) = u^*(t; x_0)$$

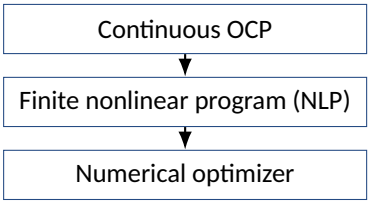


One nominal trajectory $(x_d(t), u_d(t))$. Models, disturbances, and sensing errors can cause deviation.

Two-step design: $u(t) = \underbrace{u_d(t)}_{\text{feedforward}} + \underbrace{\pi_{\text{track}}(x(t),\ x(t) - x_d(t),\ t)}_{\text{tracking feedback}}.$

Two approaches to solving optimal control problems

1. Direct methods

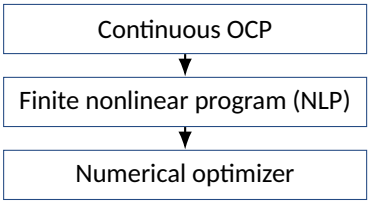


First discretize, then optimize.

- Single and multiple shooting.
- Direct collocation.

Two approaches to solving optimal control problems

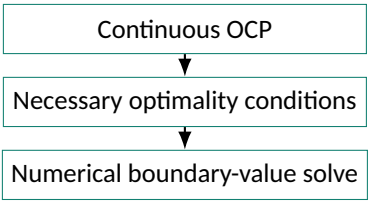
1. Direct methods



First discretize, then optimize.

- Single and multiple shooting.
- Direct collocation.

2. Indirect methods

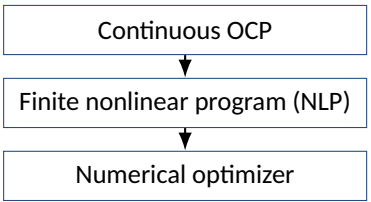


First optimize, then discretize.

- Pontryagin's Minimum Principle.
- State and costate equations; boundary conditions.

Two approaches to solving optimal control problems

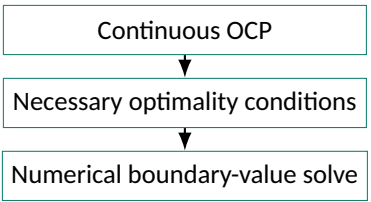
1. Direct methods



First discretize, then optimize.

- Single and multiple shooting.
- Direct collocation.

2. Indirect methods

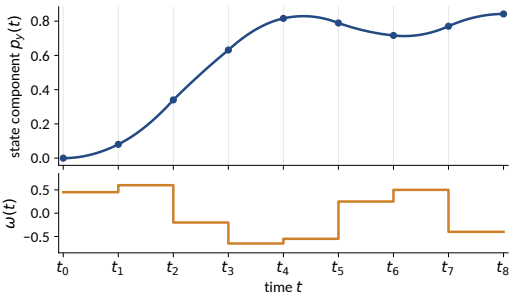


First optimize, then discretize.

- Pontryagin's Minimum Principle.
- State and costate equations; boundary conditions.

Both require numerical choices and an initial guess. We focus on **direct methods**, which interface naturally with NLP solvers.

Direct methods: Discretization



A continuous state rollout under **zero-order-hold (ZOH)** controls. State samples alone do not define the motion between them.

Choose a grid $0 = t_0 < \cdots < t_N = T$, with step Δt_k.

- ZOH: $u(t) = u_k$ on $[t_k, t_{k+1})$.
- Numerical integration defines

$$x_{k+1} = F_k(x_k, u_k).$$

- Euler: $F_k = x_k + \Delta t_k f(x_k, u_k)$.
- Alternatives: RK2/RK4, implicit or semi-implicit schemes.
- First-order hold: interpolate linearly between control values.

Discretize the cost consistently: $c_k(x_k, u_k) \approx \int_{t_k}^{t_{k+1}} c(x(t), u(t))\, dt.$

Discrete-time optimal control problem

$$
\begin{aligned}
\min_{x_0,\dots,x_N;\,u_0,\dots,u_{N-1}} \quad & \sum_{k=0}^{N-1} c_k(x_k,u_k) + h(x_N) \\
\text{subject to} \quad & x_0 = \hat{x}_0, \\
& x_{k+1} = F_k(x_k,u_k), && k = 0,\dots,N-1, \\
& x_k \in \mathcal{X}_{\mathrm{free},k}, \quad u_k \in \mathcal{U}, \\
& x_N \in \mathcal{X}_{\mathrm{free},N}.
\end{aligned}
$$

- $\hat{x}_0$: current state estimate. Decision variables are now **finitely many numbers**.
- A required goal can be added as $x_N = x_{\mathrm{goal}}$, if reachable within the horizon.
- Nonlinear cost, dynamics, or constraints generally give an **NLP**.

Optimization primer

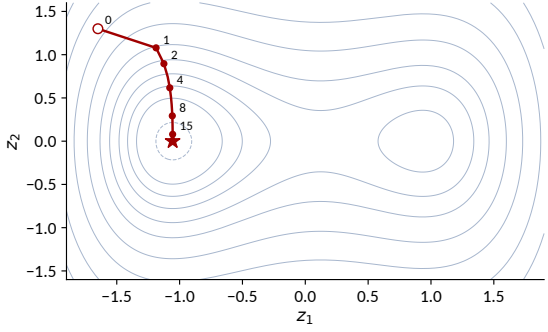
$$\min_{z} J(z) \qquad \text{subject to} \quad a(z) = 0, \quad g(z) \leq 0$$

- z: decision variables; J: objective; a, g: equality and inequality constraints.
- Begin with a guess, compute a local model, choose a step, and repeat.
- For trajopt, z will collect controls, states, or polynomial coefficients.

	Unconstrained	**Constrained**
Convex	Bowl-shaped objective	Convex objective and feasible set
Nonconvex	Multiple valleys or saddles	Dynamics, obstacles, contact

We use J, a, g for generic optimization, keeping c, h, f for the OCP. Further study: Stanford EE364A, convex optimization.

Unconstrained optimization: gradient descent



$$\min_z J(z), \qquad z^{j+1} = z^j - \alpha_j \nabla J(z^j)$$

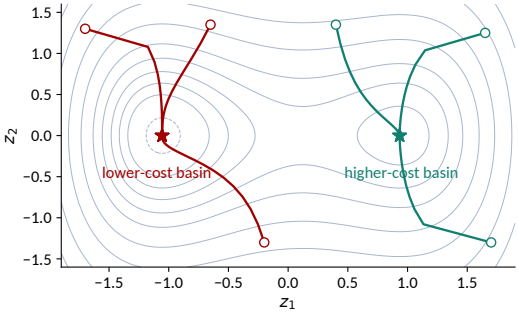
- The gradient gives the objective function's slope.
- Move downhill with step size $\alpha_j > 0$.
- $\nabla J = 0$ means **stationary**: minimum, maximum, or saddle.

Contours show equal objective values; labels count actual solver iterations.

Automatic differentiation: CasADi, JAX, and PyTorch compute derivatives.

Original computed example: $J(z) = \frac{1}{4}(z_1^2 - 1)^2 + 0.12 z_1 + \frac{1}{2} z_2^2$, step size $\alpha = 0.17$.

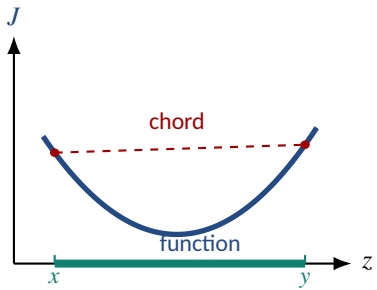
Nonconvex optimization: random restarts



1. Choose several initial guesses, e.g. random samples.
2. Run the optimizer from each guess.
3. Compare final costs.
4. Keep the best acceptable solution.

- More starts can improve the answer, but **do not certify global optimality**.
- For trajectories, different route or control guesses can expose different solutions.

Convex optimization: local minima are global



On a **convex domain**, for every x, y and $\theta \in [0, 1]$:

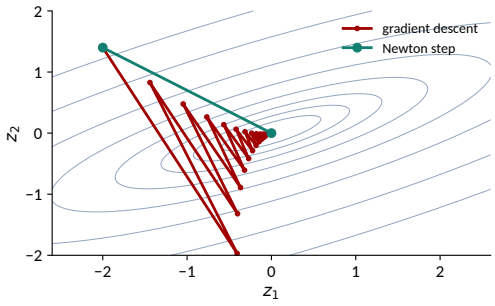
$$J\big(\theta x + (1-\theta)y\big) \leq \theta J(x) + (1-\theta)J(y).$$

The graph lies below every chord connecting two points.

- If J is twice differentiable: $\nabla^2 J(z) \succeq 0$ **everywhere** on the domain.
- Positive semidefinite means $v^\top \nabla^2 J(z) v \geq 0$ for every direction v.
- Examples: $\|z\|_2^2$, $z^\top Q z$ ($Q \succeq 0$), $\|Az - b\|_2^2$, $-\sum_i \log z_i$ ($z_i > 0$), and $-\log\det X$ ($X \succ 0$).

Every local minimizer is global. EE364A for more details.

First-order versus second-order methods

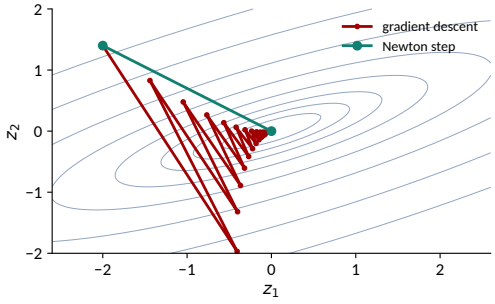


A narrow quadratic valley: gradient steps zigzag; Newton rescales directions using curvature.

A local quadratic model at z:

$$m(p) = J(z) + \nabla J(z)^\top p + \tfrac{1}{2} p^\top H p.$$

First-order versus second-order methods



A narrow quadratic valley: gradient steps zigzag; Newton rescales directions using curvature.

A local quadratic model at z:

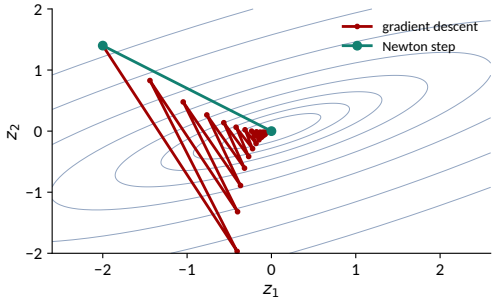
$$m(p) = J(z) + \nabla J(z)^\top p + \tfrac{1}{2} p^\top H p.$$

Newton: $H = \nabla^2 J(z)$; solve

$$Hp = -\nabla J(z), \qquad z^+ = z + \alpha p.$$

- Solve a linear system; do not form the inverse explicitly.
- Local quadratic convergence near a regular minimum, under smoothness assumptions.
- Indefinite curvature needs care: regularization, line search, or trust region.

First-order versus second-order methods



A narrow quadratic valley: gradient steps zigzag; Newton rescales directions using curvature.

A local quadratic model at z:

$$m(p) = J(z) + \nabla J(z)^\top p + \tfrac{1}{2} p^\top H p.$$

Newton: $H = \nabla^2 J(z)$; solve

$$Hp = -\nabla J(z), \qquad z^+ = z + \alpha p.$$

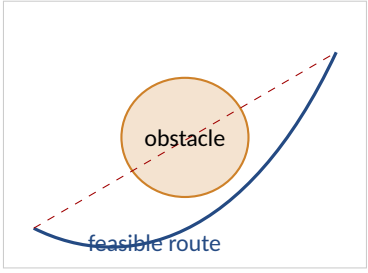
- Solve a linear system; do not form the inverse explicitly.
- Local quadratic convergence near a regular minimum, under smoothness assumptions.
- Indefinite curvature needs care: regularization, line search, or trust region.

Curvature approximations: Gauss–Newton (least squares), BFGS, L-BFGS. Momentum and Adam are first-order methods, not Hessian approximations.

Constrained optimization: feasible motion

$$\min_{z} J(z) \quad \text{s.t.} \quad a(z) = 0, \quad g(z) \le 0$$

- **Equality:** $x_{k+1} - F_k(x_k, u_k) = 0$.
- **Actuator bound:** $-u_{\max} \le u_k \le u_{\max}$.
- **Clearance:** $r^2 - \|p_k - p_{\text{obs}}\|^2 \le 0$ for a circular obstacle in position space.
- A constrained minimum need not have $\nabla J = 0$: constraints can block descent.



Convex constrained problems and quadratic programs

A standard convex formulation is

$$\min_{z} J(z) \qquad \text{s.t.} \quad Az = b, \quad g_i(z) \leq 0,$$

where J **and each** g_i **are convex**, and the equalities are **affine**.

Convex constrained problems and quadratic programs

A standard convex formulation is

$$\min_z J(z) \qquad \text{s.t.} \quad Az = b, \quad g_i(z) \le 0,$$

where J **and each** g_i **are convex**, and the equalities are **affine**. **Quadratic program (QP):**

$$\min_z \ \tfrac{1}{2} z^\top H z + b_J^\top z \quad \text{s.t.} \quad Az = b, \quad Cz \le d.$$

- Convex when $H \succeq 0$; affine constraints include bounds.
- QPs are also useful *local models* of harder problems. We will use this shortly.

A circle's exterior is nonconvex even though its interior is convex: collision avoidance is a useful counterexample.

Equality-constrained optimization: KKT equations

For $\min_z J(z)$ subject to $a(z) = 0$, introduce multipliers λ:

$$\mathcal{L}(z, \lambda) = J(z) + \lambda^\top a(z).$$

At a regular local optimum: $\nabla_z \mathcal{L} = 0$ and $\nabla_\lambda \mathcal{L} = a(z) = 0$.
A Newton step on these stationarity and feasibility equations solves

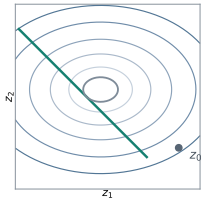
$$\underbrace{\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix}}_{\text{KKT matrix}} \begin{bmatrix} \Delta z \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ a(z) \end{bmatrix}$$

$H = \nabla^2_{zz} \mathcal{L}, \qquad A = \partial a / \partial z.$

- Top residual: stationarity error. Bottom residual: equality-constraint error.
- Update both z and λ, then repeat; a Hessian approximation is often used.
- These are **necessary conditions**, not a general certificate of a minimum.
- Linear system solves drive optimization solve cost.

Illustration

$$\min_z \quad J(z) = \tfrac{1}{2}(z - z_c)^\top Q (z - z_c), \quad Q \succ 0$$
$$\text{s.t.} \quad a(z) = c^\top z - d = 0$$



Equality-constrained optimization: KKT equations

For $\min_z J(z)$ subject to $a(z) = 0$, introduce multipliers λ:

$$\mathcal{L}(z, \lambda) = J(z) + \lambda^\top a(z).$$

At a regular local optimum: $\nabla_z \mathcal{L} = 0$ and $\nabla_\lambda \mathcal{L} = a(z) = 0$.
A Newton step on these stationarity and feasibility equations solves

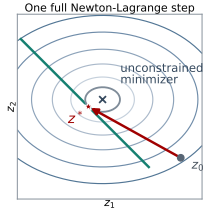
$$\underbrace{\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix}}_{\text{KKT matrix}} \begin{bmatrix} \Delta z \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} \nabla_z \mathcal{L}(z, \lambda) \\ a(z) \end{bmatrix}$$

$H = \nabla^2_{zz} \mathcal{L}, \qquad A = \partial a / \partial z.$

- Top residual: stationarity error. Bottom residual: equality-constraint error.
- Update both z and λ, then repeat; a Hessian approximation is often used.
- These are **necessary conditions**, not a general certificate of a minimum.
- Linear system solves drive optimization solve cost.

Illustration

$$\min_z \quad J(z) = \tfrac{1}{2}(z - z_c)^\top Q (z - z_c), \quad Q \succ 0$$
$$\text{s.t.} \quad a(z) = c^\top z - d = 0$$



Inequality constraints: three solver strategies

An inequality $g_i(z) \leq 0$ is **active** when $g_i(z) = 0$, and **inactive** when $g_i(z) < 0$. No advance knowledge of the active set at the minimizer.

1. **Sequential quadratic programming (SQP):** from the current guess, build a local QP (quadratic cost model, linearized constraints), solve it for a step, and repeat. Often convexified; **readily warm started**.
   Software: acados, SNOPT.
2. **Interior-point methods (IPM):** use barriers or primal-dual equations. Warm starts are often less straightforward. A workhorse of convex optimization.
   Software: IPOPT, FATROP.
3. **Augmented Lagrangian methods (ALM):** add quadratic constraint penalties to the Lagrangian; alternate primal minimization and multiplier updates.
   Software: ALTRO, proxNLP.

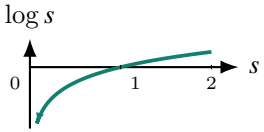
No universal winner: Try all three. Warm starting reuses a previous solution and solver information.

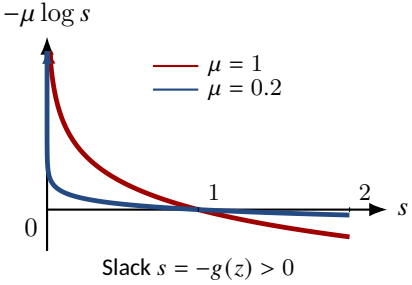
A useful workflow: IPM to convergence initially $\longrightarrow$ warm-started SQP/ALM with limited iterations for real-time control.

Interior-point methods: the log-barrier idea

For inequalities $g_i(z) \le 0$, add a barrier to the objective:

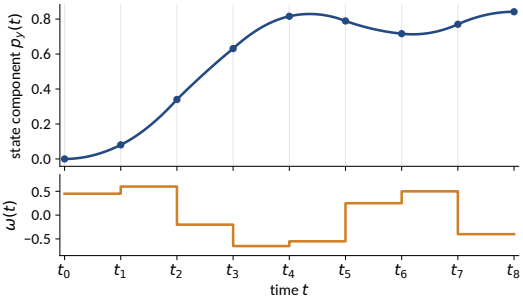
$$\min_z \quad J(z) - \mu \sum_i \log[-g_i(z)] \qquad \text{s.t.} \quad a(z) = 0.$$





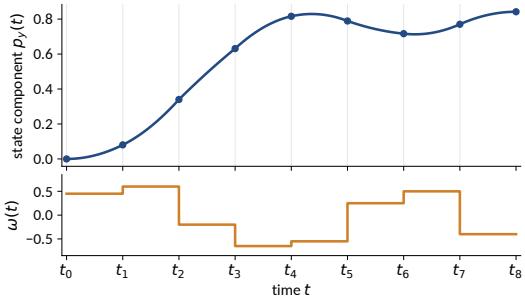
- The log is defined only for $g_i(z) < 0$: this is the barrier's **domain**.
- The barrier tends to $+\infty$ as a constraint boundary is approached.
- Solve, reduce $\mu > 0$, and repeat to approach the constrained solution.

Direct method: single shooting



Controls are held constant in each interval; the state follows one continuous rollout from x_0.

Direct method: single shooting



Controls are held constant in each interval; the state follows one continuous rollout from x_0.

Eliminate states by substitution:

$$x_1 = F_0(x_0, u_0), \quad x_2 = F_1(F_0(x_0, u_0), u_1).$$

Write the rollout as $X_k(U)$, where $U = (u_0, \ldots, u_{N-1})$ and $X_0 = x_0$:

$$\min_U \quad \sum_{k=0}^{N-1} c_k(X_k(U), u_k) + h(X_N(U))$$

$$\text{s.t.} \quad X_k(U) \in \mathcal{X}_{\text{free},k},$$
$$u_k \in \mathcal{U}, \quad X_N(U) \in \mathcal{X}_{\text{free},N}.$$

- Only controls are independent decision variables; dynamics equalities disappear.
- State-dependent costs, safety constraints, and any required endpoint remain.

Single shooting: four drawbacks

A long chain of sensitivities

$$\frac{\partial x_N}{\partial u_0} = A_{N-1} \cdots A_1 B_0$$

$A_k = \partial F_k / \partial x_k$, $B_k = \partial F_k / \partial u_k$. For $x_{k+1} = a x_k + u_k$:

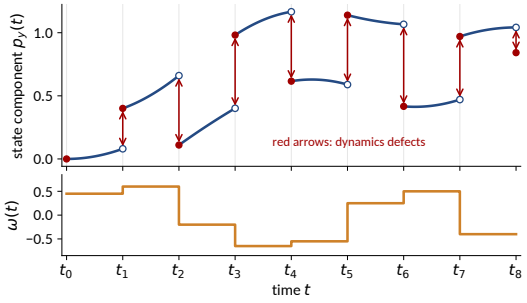
$$\frac{\partial x_N}{\partial u_0} = a^{N-1}.$$

$|a| > 1$: growth.
$|a| < 1$: decay.

1. **Conditioning:** long products can amplify or suppress gradients, especially over long horizons or unstable dynamics.
2. **Sequential rollout:** each next state needs the previous simulated state. Parallelism across time is restricted.
3. **Dense derivatives:** an early control affects many later costs and constraints.
4. **Restricted initial guesses:** state guesses cannot be independent of the controls and their rollout.

Still useful: compact problems, manageable horizons, and rollout-based methods such as basic iLQR.

Direct method: multiple shooting



An **infeasible iterate**: filled red nodes are independent state guesses; open blue nodes are local rollout endpoints. Red arrows show defects.

Keep states and controls as variables:

$$\min_{X,U} \quad \sum_{k=0}^{N-1} c_k(x_k, u_k) + h(x_N)$$

$$\text{s.t.} \quad x_0 = \hat{x}_0,$$

$$\underbrace{x_{k+1} - F_k(x_k, u_k)}_{d_k\text{: dynamics defect}} = 0,$$

$$x_k \in \mathcal{X}_{\text{free},k}, \quad u_k \in \mathcal{U},$$

$$x_N \in \mathcal{X}_{\text{free},N}.$$

$X = (x_0, \ldots, x_N)$; $U = (u_0, \ldots, u_{N-1})$.

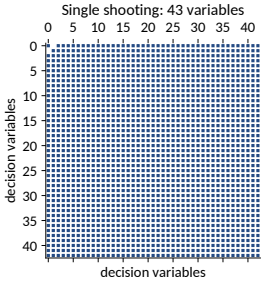
Local simulation, global consistency: integrate each interval separately; the optimizer drives all defects to zero.

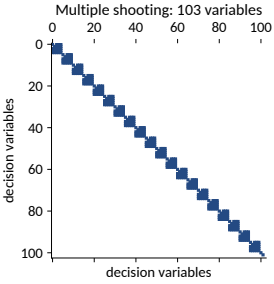
Multiple shooting: more variables, useful structure

1. **Shorter derivative chains:** local interval sensitivities can improve conditioning.
2. **Parallel local work:** interval simulation, cost evaluation, and derivatives can be evaluated independently at a given iterate.
3. **Sparse derivatives:** Constraints and cost expressions involve only a single time step or neighboring time steps,
4. **Flexible initialization:** initialize states from a geometric path even if the initial dynamics defects are nonzero.

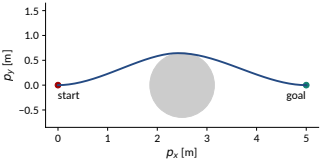
The NLP is larger, but the sparsity structure is more favorable.

An AV example: Hessian sparsity





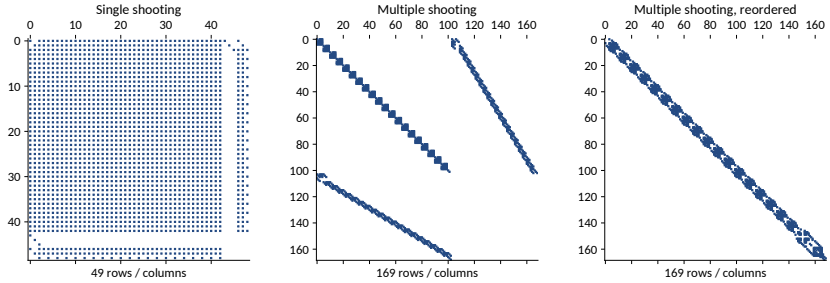
$H = \nabla^2_{zz}\mathcal{L}$: symbolic structural nonzeros.
SS: 1,847 nonzeros. MS: 462 nonzeros despite more variables.



Unicycle kinematic model:
$\dot{x} = (v\cos\theta,\ v\sin\theta,\ \omega)^\top$,
$x = (p_x, p_y, \theta)$.

$N = 20$, $T = 6$ s; RK4. Bounds on v, ω; circular obstacle; fixed endpoints.

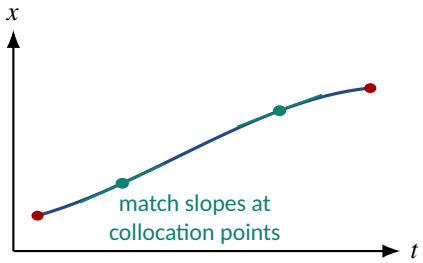
KKT sparsity: ordering exposes time structure



$$K_{\text{eq}} = \begin{bmatrix} H & A_{\text{eq}}^{\top} \\ A_{\text{eq}} & 0 \end{bmatrix}.$$

- Larger MS system, but fewer nonzeros and local couplings.
- Simultaneously permuting rows and columns exposes a band: the problem is unchanged.

Direct collocation: optimize a polynomial trajectory

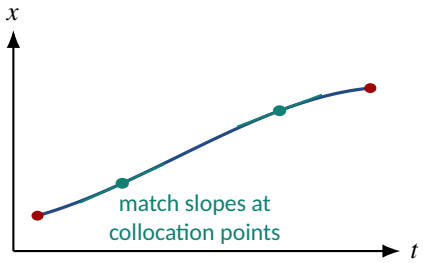


Examples: Hermite–Simpson, Gauss/Radau.

Tools: Rockit, GPOPS-II.

- **Represent:** a polynomial state $p_k(t)$ on each interval; optimize its coefficients or nodal values, together with controls.

Direct collocation: optimize a polynomial trajectory



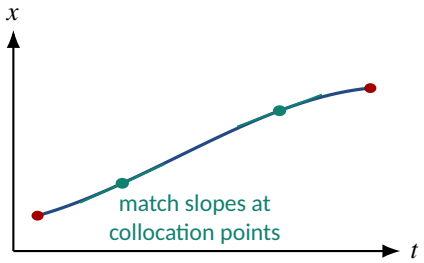
Examples: Hermite–Simpson, Gauss/Radau.

Tools: Rockit, GPOPS-II.

- **Represent:** a polynomial state $p_k(t)$ on each interval; optimize its coefficients or nodal values, together with controls.
- **Match dynamics** at collocation points:

$$\dot{p}_k(t_{k,j}) = f(p_k(t_{k,j}), u(t_{k,j})).$$

Direct collocation: optimize a polynomial trajectory



- **Represent:** a polynomial state $p_k(t)$ on each interval; optimize its coefficients or nodal values, together with controls.
- **Match dynamics** at collocation points:

$$\dot{p}_k(t_{k,j}) = f(p_k(t_{k,j}), u(t_{k,j})).$$

- **Join intervals:** enforce state continuity; add costs and path/boundary constraints.

Examples: Hermite–Simpson, Gauss/Radau.

Tools: Rockit, GPOPS-II.

Local coupling gives a sparse NLP. Collocation schemes can suit stiff dynamics.

Caveat: check dynamics and constraints between nodes; refine the mesh as needed.

M. Kelly (2017), *An Introduction to Trajectory Optimization*, SIAM Review, DOI: 10.1137/16M1062569.

A special case: linear quadratic regulator (LQR)

$$\begin{aligned} \min_{u_0,\ldots,u_{N-1}} \quad & x_N^\top Q_f x_N + \sum_{k=0}^{N-1} \left(x_k^\top Q x_k + u_k^\top R u_k\right) \\ \text{subject to} \quad & x_{k+1} = Ax_k + Bu_k, \qquad x_0 = \hat{x}_0. \end{aligned}$$

1. **Convex quadratic cost:** $Q, Q_f \succeq 0$, $R \succ 0$.
2. **Linear dynamics.**
3. **No state/input inequalities:** dynamics are still equality constraints.

- Closed-form recursion gives a feedback policy $u_k = -K_k x_k$.
- **Kalman (1960):** a foundational optimal-control result, with discrete and continuous-time versions.
- Widely used today as a **controller and computational building block** in nonlinear trajectory optimization. See **AA212**.

R. E. Kalman, "Contributions to the Theory of Optimal Control" (1960).

LQR solution: backward Riccati recursion

The exact optimal cost-to-go is $V_k(x) = x^\top P_k x$. Initialize $P_N = Q_f$:

LQR solution: backward Riccati recursion

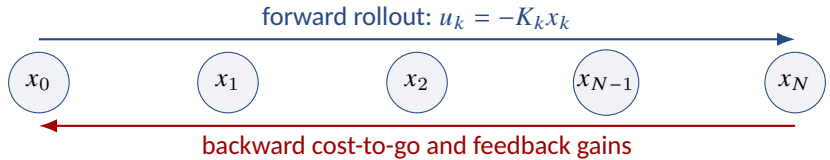
The exact optimal cost-to-go is $V_k(x) = x^\top P_k x$. Initialize $P_N = Q_f$:

$$
\begin{aligned}
S_k &= R + B^\top P_{k+1} B, \\
K_k &= S_k^{-1} B^\top P_{k+1} A, \\
P_k &= Q + A^\top P_{k+1} A - A^\top P_{k+1} B K_k, \qquad k = N-1, \ldots, 0.
\end{aligned}
$$

LQR solution: backward Riccati recursion

The exact optimal cost-to-go is $V_k(x) = x^\top P_k x$. Initialize $P_N = Q_f$:

$$
\begin{aligned}
S_k &= R + B^\top P_{k+1} B, \\
K_k &= S_k^{-1} B^\top P_{k+1} A, \\
P_k &= Q + A^\top P_{k+1} A - A^\top P_{k+1} B K_k, \qquad k = N-1, \ldots, 0.
\end{aligned}
$$



- For fixed state/input dimensions, the recursion scales linearly in horizon length.
- Solving for each gain uses linear algebra; implement a solve rather than an explicit inverse.
- LQR gives both a nominal solution and feedback gains for other initial states.

Infinite-horizon LQR: a Riccati fixed point

For time-invariant dynamics and weights:

$$\min_{u_0,u_1,\dots} \sum_{k=0}^{\infty} (x_k^\top Q x_k + u_k^\top R u_k), \qquad x_{k+1} = Ax_k + Bu_k.$$

The stabilizing fixed point satisfies the **discrete algebraic Riccati equation**:

$$P = Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A.$$

Then

$$K = (R + B^\top P B)^{-1} B^\top P A, \qquad u = -Kx, \qquad x_{k+1} = (A - BK)x_k.$$

- With $Q \succeq 0$, $R \succ 0$, (A, B) stabilizable and $(Q^{1/2}, A)$ detectable, the stabilizing solution yields asymptotic stability.
- Stabilizable: unstable modes can be controlled. Detectable: unpenalized modes must already be stable.
- Finite-horizon LQR with an arbitrary terminal cost does not automatically provide this infinite-horizon guarantee.

Motivating iLQR: quadratic cost, nonlinear dynamics

For now, omit state and control inequalities and consider

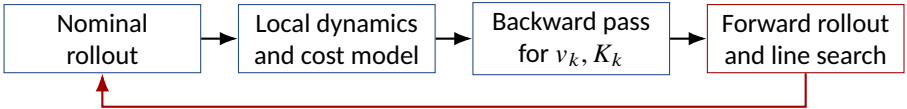
$$\begin{aligned}\min_{U} \quad & (x_N - x_N^{\text{ref}})^\top Q_f (x_N - x_N^{\text{ref}}) \\ & + \sum_{k=0}^{N-1} \left[(x_k - x_k^{\text{ref}})^\top Q (x_k - x_k^{\text{ref}}) + (u_k - u_k^{\text{ref}})^\top R (u_k - u_k^{\text{ref}}) \right] \\ \text{subject to} \quad & x_{k+1} = F_k(x_k, u_k), \qquad x_0 = \hat{x}_0.\end{aligned}$$

- This resembles LQR, but the dynamics F_k are **nonlinear**.
- Linearize dynamics around a nominal rollout $(\bar{x}_k, \bar{u}_k)$:

$$\delta x_{k+1} \approx A_k \delta x_k + B_k \delta u_k, \quad A_k = F_{k,x},\ B_k = F_{k,u}.$$

- Solve the local linear-quadratic problem efficiently, update the trajectory, and repeat.

iLQR: reuse the backward recursion



$$u_k^{\text{new}} = \bar{u}_k + \alpha v_k - K_k(x_k^{\text{new}} - \bar{x}_k).$$

- v_k: feedforward increment; K_k: feedback gain; α: step length.
- Basic rollout-based iLQR maintains dynamics feasibility: single shooting.
- Handling constraints require extensions.

Tassa, Erez, and Todorov (2012), *Synthesis and Stabilization of Complex Behaviors through Online Trajectory Optimization*.

Constrained solvers that exploit OCP structure

Examples	Approach	Why useful
acados, MIM Solvers	SQP / related sequential methods	Local constrained subproblems with structured linear algebra; warm starts and online control.
FATROP Interactive playground	Nonlinear interior point	Generalized Riccati factorization; supports path equalities; demonstrated on challenging minimum-time OCPs.
ALTRO	Augmented Lagrangian + iLQR	Fast trajectory subproblems with multiplier/penalty updates and constraint refinement.
proxNLP	Primal-dual augmented Lagrangian	Nonlinear programming for robotics and beyond.

- Speed comes from **structure exploitation**, derivatives, and implementation choices.
- Compare on matched dynamics, constraints, horizons, tolerances, and hardware.

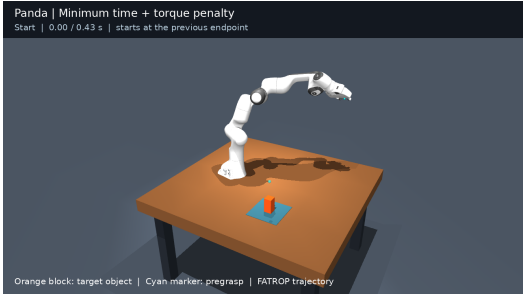
Sources: acados documentation; FATROP paper; MIM Solvers; ALTRO; proxNLP.

FATROP example: minimum-time robot motion

7-DOF Panda: joint-space double integrator, inverse-dynamics torque limits.

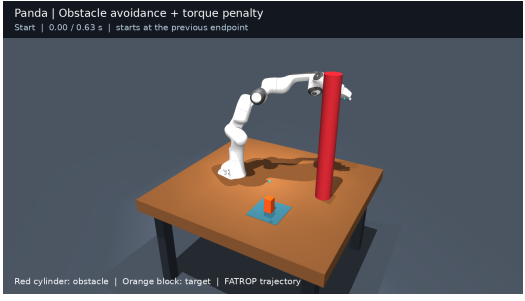
$$\min \sum_{k=0}^{N-1} \Delta t_k \left(1 + \frac{10^{-4}}{2} \|\tau_k\|^2\right), \qquad N = 60, \quad \Delta t_{k+1} = \Delta t_k.$$

Reach a pregrasp pose



Motion: **0.435 s** Median solve: **245 ms**

Avoid a cylindrical obstacle



Motion: **0.628 s** Median solve: **390 ms**

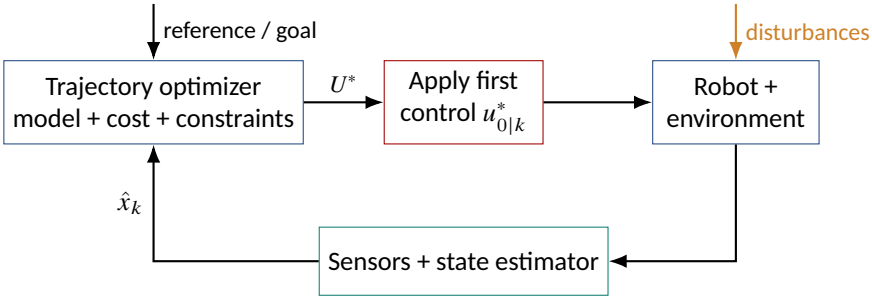
Software: model, differentiate, transcribe, solve



- **CasADi:** symbolic expression graphs, algorithmic differentiation, integrators, and interfaces to solvers. JAX/PyTorch provide related differentiation tools.
- **Rockit:** high-level OCP modeling and direct transcription on top of CasADi; shooting, collocation, and solver interfaces.
- **GPOPS-II:** hp-adaptive pseudospectral optimal control.
- **General NLP solvers:** SNOPT (SQP), IPOPT (IPM), KNITRO (several methods), proxNLP (primal-dual ALM).
- **Structured OCP solvers:** the preceding slide. Packages differ in their support for path equalities, constraints, and code generation.

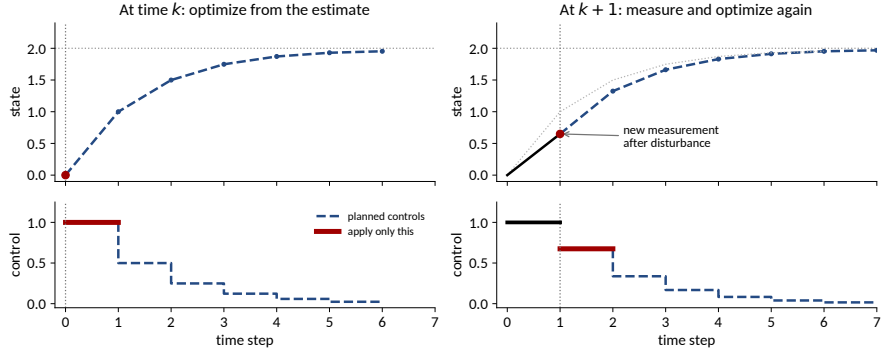
https://web.casadi.org/; https://meco-software.pages.gitlab.kuleuven.be/rockit/; https://github.com/Simple-Robotics/proxsuite-nlp.

Model predictive control: close the loop with trajopt



- Solve from the latest estimate, apply only the first action, then shift the horizon.
- Replanning turns repeated open-loop optimization into a feedback policy.
- Reuse the shifted solution as a warm start; the solve must meet its deadline.

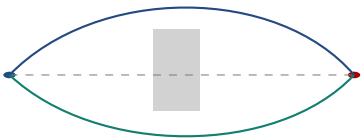
MPC: execute one action, measure, and replan



1. At k, optimize a horizon of future controls; execute the red first input.
2. A disturbance moves the state away from the prediction. Obtain a **new measurement**.
3. At $k+1$, solve from this state; execute the first input of the **new** plan.

Limitations and frontiers

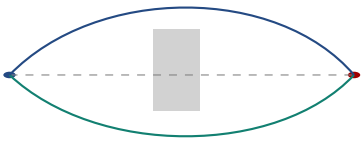
Route choice can trap local optimization



Goal-only trajopt can work locally; longer routes may need graph search, sampling, or multiple initial guesses.

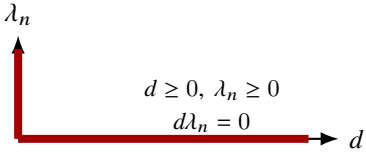
Limitations and frontiers

Route choice can trap local optimization



Goal-only trajopt can work locally; longer routes may need graph search, sampling, or multiple initial guesses.

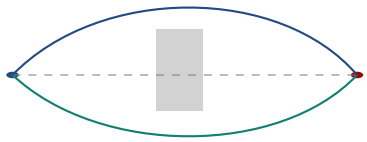
Contact introduces discrete choices



Gap d and normal force λ_n cannot both be positive: a mathematical program with complementarity constraints (MPCC).

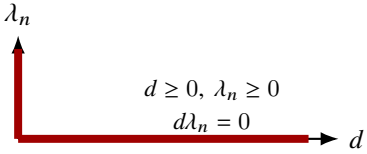
Limitations and frontiers

Route choice can trap local optimization



Goal-only trajopt can work locally; longer routes may need graph search, sampling, or multiple initial guesses.

Contact introduces discrete choices



Gap d and normal force λ_n cannot both be positive: a mathematical program with complementarity constraints (MPCC).

- **Logic + motion:** jointly choose routes, grasps, and contact modes. Non-smooth optimization.
- **Global guarantees:** GCS and polynomial optimization offer model-specific relaxations/certificates, not a general solution to nonlinear OCPs.
- **Speed + learning:** faster structured solves, differentiable optimization, trajectory data, and learned warm starts.

GCS example: Marcucci et al., *Motion Planning around Obstacles with Convex Optimization*, `https://arxiv.org/abs/2205.04422`.

Conclusions: put the pieces back into the autonomy stack



What becomes possible

- Dynamic/kinematic feasibility, actuator limits, and obstacle avoidance in one model.
- Tradeoffs among time, effort, tracking, and clearance.
- Aggressive motion and online replanning with structured solvers.

What still needs attention

- Nonconvex local minima and poor initial guesses.
- Discretization error, solver tolerances, and computation deadlines.
- Inaccurate models, uncertain perception, and execution disturbances.

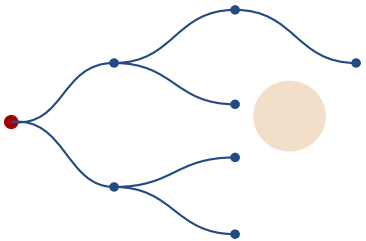
Next: trajectory tracking and feedback control. A plan is useful when the robot can follow it.

References and next steps for self-study

- **Optimal control:** D. E. Kirk, *Optimal Control Theory: An Introduction* (Dover reprint, 2004; original 1970). Stanford **AA203**.
- **Theory:** D. Liberzon, *Calculus of Variations and Optimal Control Theory: A Concise Introduction* (2012).
- **Optimization:** Boyd and Vandenberghe, *Convex Optimization* (2004); Stanford **EE364A**.
- **Direct methods:** M. Kelly, *An Introduction to Trajectory Optimization: How to Do Your Own Direct Collocation* (SIAM Review, 2017).
- **Feedback and LQR:** Stanford **AA212**; Kalman (1960). **iLQR/DDP:** Tassa, Erez, and Todorov (IROS, 2012).
- **Autonomy stack:** Paden et al. (2016), motion planning and control for self-driving urban vehicles.

Software and frontier sources appear on the relevant slides. The course references identify subjects; consult the current catalog for offerings.

Optional: kinodynamic planning



- Search in **state space**, respecting differential constraints along every edge.
- Sample controls and durations; integrate $\dot{x} = f(x, u)$ to create candidate motions.
- Reject collisions and violated limits; grow a tree or graph of feasible motions.
- Useful for exploring route alternatives and providing initial guesses for trajectory optimization.

Geometric planners, kinodynamic planners, and trajopt can be combined. A path planner need not ignore dynamics; that is an architectural choice.

Original schematic. Kinodynamic RRT background: LaValle and Kuffner, *Randomized Kinodynamic Planning* (2001).

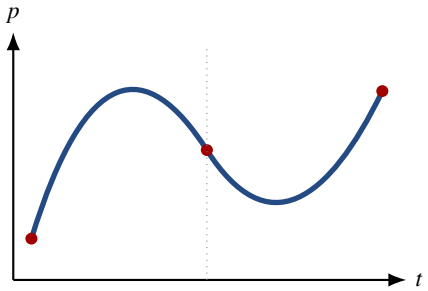
Optional: kinodynamic RRT and optimality

1. Sample a target state x_{rand} and choose a nearby tree state x_{near}.
2. Choose/sample an input $u \in \mathcal{U}$ and duration Δt.
3. Propagate $x_{\mathrm{new}} = F_{\Delta t}(x_{\mathrm{near}}, u)$.
4. Add the edge if the *whole motion* satisfies collision and system constraints.

- Distance metrics are difficult: Euclidean proximity need not imply easy dynamic reachability.
- Steering between two specified states can itself require solving an OCP.
- RRT can be probabilistically complete under assumptions; standard RRT is not asymptotically optimal.
- RRT* adds cost-based parent selection and rewiring. Extending its guarantees to dynamics requires appropriate steering/reachability assumptions; arbitrary rollout-and-rewire is insufficient.

Karaman and Frazzoli (2011), *Sampling-based Algorithms for Optimal Motion Planning*; Kleinbort et al. (2018), https://arxiv.org/abs/1809.07051.

Optional: splines as a trajectory representation



On segment i, with local time τ:

$$p_i(\tau) = \sum_{j=0}^{d} a_{ij}\tau^j.$$

- Coefficients define position and its time derivatives analytically.
- Endpoint and continuity conditions connect neighboring pieces.
- Degree d gives $d + 1$ scalar coefficients per segment per coordinate.
- Optimize coefficients and, if needed, segment durations.

Smoothness and feasibility are separate. A smooth curve still needs dynamic/kinematic, actuator, and collision checks.

Optional: coefficients and boundary conditions

For a cubic $p(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$ on $[0, T]$, prescribe position and velocity at both endpoints:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & T & T^2 & T^3 \\ 0 & 1 & 2T & 3T^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} p_0 \\ v_0 \\ p_T \\ v_T \end{bmatrix}.$$

- For $T > 0$, four independent conditions determine these four coefficients.
- A quintic has six coefficients and can additionally match endpoint accelerations.
- With more unknowns than independent conditions, an objective selects among feasible splines.
- Piecewise trajectories also need continuity of the required derivatives at joins: $p_i^{(r)}(T_i) = p_{i+1}^{(r)}(0)$.

Optional: minimum-snap trajectory planning

For polynomial position $p_a(t)$ with coefficients a:

$$\min_a \int_0^T \left\| \frac{d^4 p_a(t)}{dt^4} \right\|^2 dt \qquad \text{s.t. waypoint and continuity constraints.}$$

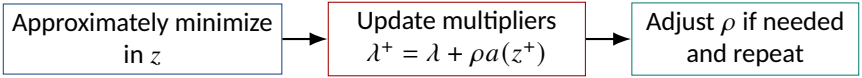
- Snap is the fourth time derivative of position. Penalizing it discourages abrupt changes in high-order motion.
- With **fixed segment times**, the integral is quadratic in the coefficients; endpoint/continuity constraints are linear. This gives a convex QP with suitable affine constraints.
- Optimizing time allocation or adding nonconvex obstacles generally changes that convexity.
- Quadrotor differential flatness connects position/yaw derivatives to states and inputs under model assumptions.
- Validate thrust/torque limits, tracking, and collision clearance along the whole trajectory.

Mellinger and Kumar (ICRA 2011), *Minimum Snap Trajectory Generation and Control for Quadrotors*, DOI: 10.1109/ICRA.2011.5980409.

Optional: Augmented Lagrangian methods

For equalities $a(z) = 0$, use

$$\mathcal{L}_\rho(z, \lambda) = J(z) + \lambda^\top a(z) + \frac{\rho}{2}\|a(z)\|^2, \qquad \rho > 0.$$



- Inequalities need a projected variant or slack variables; the displayed formula is equality-only.
- Warm starts and efficient inner solves can be useful for online refinement.
- Large penalties can worsen conditioning; stringent feasibility tolerances may be costly.
- An infeasible problem has no feasible answer: **check residuals**, regardless of solver family.

Examples: ALTRO and proxNLP; their augmented-Lagrangian methods differ in design.

Optional: sequential quadratic programming (SQP)

SQP solves a nonlinear constrained problem by repeatedly building and solving a local quadratic program (QP).

1. Linearize equality and inequality constraints near the current iterate.
2. Approximate the objective with a quadratic model; the curvature term is often regularized.
3. Solve the QP for a proposed step, choose a step length, and update the iterate.
4. Rebuild the local model and repeat until optimality and constraint residuals meet the chosen tolerances.

Warm starts can reuse shifted states, controls, and multipliers in repeated problems. SQP often benefits when successive problems are similar.

SNOPT is a general SQP solver; KNITRO and acados also offer SQP-based options.

Optional: the SQP quadratic subproblem

At iterate z^j, SQP solves a QP in the proposed step p:

$$\begin{aligned} \min_{p} \quad & \nabla J(z^j)^\top p + \tfrac{1}{2} p^\top B_j p \\ \text{s.t.} \quad & a(z^j) + A_j p = 0, \\ & g(z^j) + G_j p \le 0. \end{aligned}$$

- B_j is the Lagrangian Hessian or a curvature approximation; it is often regularized.
- A_j and G_j are the equality and inequality constraint Jacobians.
- The QP gives a direction, not the final answer: take a suitable step $z^{j+1} = z^j + \alpha_j p$, then rebuild the local model.
- For nonconvex problems SQP is a local method; globalization and feasibility handling matter.